

TECHNICAL PROJECT REPORT

High-Precision Tilt Measurement System

Using ADXL355 MEMS Accelerometer & STM32 Microcontroller

EVAL-ADXL355Z Evaluation Board • SPI Interface • Structural Health Monitoring

Sponsored by

TDCoB Pvt. Ltd.

Automation & IoT-Based Monitoring Systems

Project Duration: March 2026 – May 2026 | Rev. 2: Hardware + Programming Guide Added

Institute	Vishwakarma Institute of Technology, Pune
Name of Students	Aakash Dabhade Dhruvi Indani Manushree Jadhav
Project Duration	March 2026 – May 2026
Guidance	Dr. Ketan Raut & Dr. Minal Deshmukh

1. Introduction

Structural Health Monitoring (SHM) is a critical discipline in modern engineering used to assess the integrity and stability of civil and industrial structures such as bridges, buildings, dams, and large-scale installations. One of the most vital parameters tracked in SHM is tilt or inclination, which enables early detection of deformation, displacement, or structural imbalance before catastrophic failure occurs.

This report presents the complete development of a high-precision tilt measurement system targeting angular resolution as fine as $\pm 0.01^\circ$. The system uses the ADXL355 digital MEMS accelerometer mounted on the EVAL-ADXL355Z evaluation board from Analog Devices, interfaced to an STM32 microcontroller via SPI. This revision adds detailed hardware wiring, STM32CubeMX configuration, and full firmware implementation guidance based on the official EVAL-ADXL355Z User Guide (UG-1030).

2. Industry Collaboration

Sponsoring Organization	TDCoB Pvt. Ltd.
Domain	Automation & IoT-Based Monitoring Systems
Objective	Develop a high-accuracy inclinometer for industrial use
Project Duration	March 2026 – May 2026
Guidance	Pranay Upadhyay

3. System Overview

3.1 Key Components

- EVAL-ADXL355Z Evaluation Board (Analog Devices, UG-1030) — primary sensing platform
- STM32 Microcontroller (Nucleo-F446RE or equivalent) — data acquisition and processing
- SPI Communication Interface (Mode 0, CPOL=0, CPHA=0)
- Calibration and Digital Filtering Algorithms (firmware)
- Output Interface — UART serial terminal or OLED/LCD display

3.2 End-to-End Data Flow

- 1. ADXL355 measures gravity projection on X, Y, Z axes (20-bit resolution)
- 2. Raw counts transmitted to STM32 via SPI — Header P2 of EVAL-ADXL355Z
- 3. STM32 converts raw counts to acceleration in g using scale factor
- 4. Offset calibration applied; temperature compensation calculated
- 5. Low-pass filter and averaging reduce measurement noise
- 6. Tilt angles computed via atan2() trigonometry
- 7. Results streamed via UART or displayed on screen

4. Sensor Selection — ADXL355

The ADXL355 3-axis digital MEMS accelerometer by Analog Devices is selected as the primary sensing element. Its specifications directly satisfy the sub-degree tilt resolution requirements of this project.

Feature	Specification / Benefit
Noise Density	~22.5 $\mu\text{g}/\sqrt{\text{Hz}}$ — ultra-low for precision sensing
ADC Resolution	20-bit signed output per axis
Temperature Stability	Low drift, suitable for outdoor SHM deployments
Interface	Digital SPI (also supports I2C) — immune to analog noise
Measurement Range	$\pm 2\text{ g}$ / $\pm 4\text{ g}$ / $\pm 8\text{ g}$ — selectable via register
Supply Voltage	3.3 V (VDDIO) — compatible with STM32 I/O
Output Data Rate	4 Hz to 4000 Hz — programmable

5. EVAL-ADXL355Z Evaluation Board — Hardware Details

The EVAL-ADXL355Z (Analog Devices User Guide UG-1030) is a compact 0.8 in × 0.8 in evaluation board designed for rapid prototyping with the ADXL355. Understanding its hardware is essential for correct wiring to the STM32.

5.1 Board Features

- Two 6-pin headers (P1 and P2) providing access to all ADXL355 pins
- Decoupling capacitors (0.1 μ F, 50 V, X7R ceramic) on all supply lines for noise rejection
- Series resistors (1 k Ω) on V1P8ANA and V1P8DIG pins for isolation
- STBY jumper for standby mode control
- Four corner mounting holes for mechanical attachment to structure or PCB
- Board stiffness and small footprint minimize mechanical influence on measurements

5.2 Header P1 — Power and Interrupt Pins

Header P1 provides access to the following signals (per UG-1030, Figure 5):

P1 Pin	Signal	Direction	Description
1	VDDIO	Power In	Digital I/O supply — connect to 3.3 V
2	VDD	Power In	VSUPPLY — main power, connect to 3.3 V
3	VSS/VSSIO	Ground	Ground reference
4	INT1	Output	Interrupt 1 — DRDY, FIFO, activity alerts
5	INT2	Output	Interrupt 2 — configurable interrupt
6	DRDY	Output	Data Ready — pulses when new data available

5.3 Header P2 — SPI / I2C Communication Pins

Header P2 carries the SPI interface signals used to communicate with the STM32 (per UG-1030, Figure 6):

P2 Pin	Signal	STM32 Pin	Description
1	V1P8ANA	NC (or 1.8V reg)	Analog LDO supply (internal or external 1.8V)
2	V1P8DIG	NC (or 1.8V reg)	Digital LDO supply (internal or external 1.8V)
3	MISO / ASEL	SPI MISO (e.g. PA6)	Master In Slave Out — data from ADXL355 to STM32
4	CS / SCL	GPIO Out (e.g. PA4)	Chip Select — active LOW to select ADXL355

5	SCLK / VSSIO	SPI SCLK (e.g. PA5)	Serial Clock from STM32 master
6	MOSI / SDA	SPI MOSI (e.g. PA7)	Master Out Slave In — data from STM32 to ADXL355

WARNING: The EVAL-ADXL355Z is NOT reverse-polarity protected. Reversing VSS or VDDIO can permanently damage the ADXL355. Dropping the board onto a hard surface can generate thousands of g, exceeding absolute maximum limits.

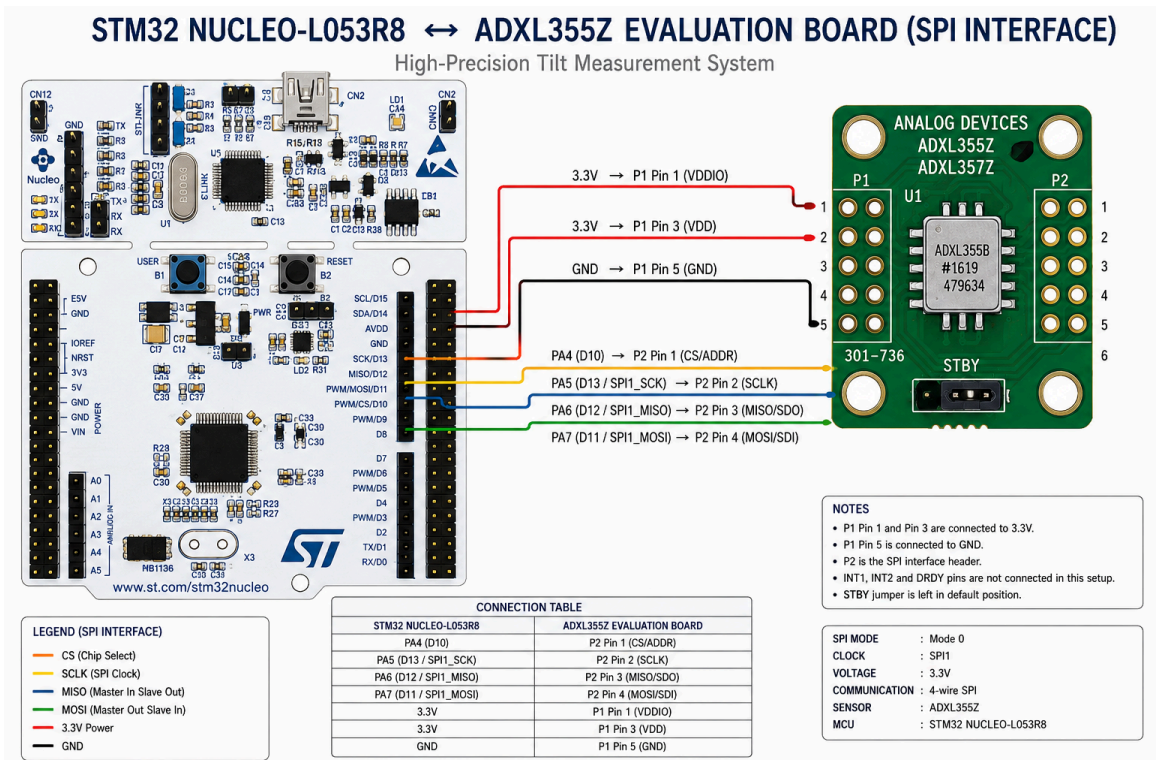


Figure 1: Complete Hardware Wiring Diagram between STM32 NUCLEO-L053R8 and ADXL355Z Evaluation Board using SPI Interface.

6. Wiring — EVAL-ADXL355Z to STM32

The EVAL-ADXL355Z board connects to the STM32 Nucleo board using six signal wires. Power is sourced from the STM32 Nucleo 3.3 V pin. The connection table below uses STM32 Nucleo-F446RE pin names as reference; equivalent pins on other Nucleo boards may be used.

EVAL-ADXL355Z Pin	Signal	STM32 Nucleo Pin	Notes
P1 Pin 1 (VDDIO)	3.3 V Power	+3V3 (CN6)	Must be 3.3 V, NOT 5 V

P1 Pin 2 (VDD)	3.3 V Power	+3V3 (CN6)	Both VDD and VDDIO to 3.3 V
P1 Pin 3 (VSS)	Ground	GND (CN6)	Common ground reference
P2 Pin 6 (MOSI)	SPI MOSI	PA7 / D11 (CN5)	STM32 SPI1 MOSI
P2 Pin 3 (MISO)	SPI MISO	PA6 / D12 (CN5)	STM32 SPI1 MISO
P2 Pin 5 (SCLK)	SPI Clock	PA5 / D13 (CN5)	STM32 SPI1 SCK
P2 Pin 4 (CS)	Chip Select	PA4 / D7 (CN9)	GPIO Output, active LOW, software controlled
P1 Pin 6 (DRDY)	Data Ready	PB0 / D3 (CN9)	Optional — GPIO Input with interrupt

NOTE: V1P8ANA (P2 Pin 1) and V1P8DIG (P2 Pin 2) are left unconnected when using the ADXL355's integrated LDOs (powered from VDD). Only connect these if using external 1.8 V regulated supplies.

7. STM32CubeMX Configuration

STM32CubeMX is used to graphically configure the microcontroller peripherals and generate the HAL-based initialization code. Follow the steps below to configure the project correctly.

7.1 Create New Project

8. Open STM32CubeMX and click 'New Project'
9. In the MCU Selector, search for your board (e.g., NUCLEO-F446RE)
10. Select your board and click 'Start Project'
11. When prompted, accept default peripheral initialization

7.2 SPI1 Peripheral Configuration

Navigate to Connectivity → SPI1 in the left panel:

CubeMX Parameter	Value to Set
Mode	Full-Duplex Master
Hardware NSS Signal	Disabled (software CS via GPIO)
Frame Format	Motorola
Data Size	8 Bits
First Bit	MSB First
Clock Polarity (CPOL)	Low (CPOL = 0)
Clock Phase (CPHA)	1 Edge (CPHA = 0) → SPI Mode 0
Prescaler (Baud Rate)	Divide by 16 → ~5.625 MHz (adjust to ≤8 MHz max)

7.3 GPIO Configuration for Chip Select

Navigate to System Core → GPIO:

12. Find PA4 in the pin table — set to GPIO_Output
13. Set GPIO output level: High (CS is active LOW, idle HIGH)
14. Set GPIO mode: Output Push-Pull
15. No pull-up or pull-down required (sensor has internal pull-up on CS)
16. User Label: ADXL355_CS

7.4 USART2 for Debug Output

Navigate to Connectivity → USART2:

17. Mode: Asynchronous
18. Baud Rate: 115200 bps
19. Word Length: 8 Bits, No Parity, 1 Stop Bit
20. This enables printf() output to ST-Link virtual COM port

7.5 Clock Configuration

21. Navigate to Clock Configuration tab
22. Set HCLK to maximum (180 MHz for F446RE)
23. Ensure APB2 peripheral clock (for SPI1) is correctly derived
24. CubeMX auto-resolves clock tree conflicts

7.6 Generate Code

25. Go to Project Manager → enter project name and location
26. Toolchain/IDE: STM32CubeIDE
27. Click 'Generate Code' — CubeMX creates the full HAL project
28. Click 'Open Project' to launch STM32CubeIDE

8. ADXL355 Key Registers

The ADXL355 is configured and read via its SPI register interface. The following registers are critical for initialization and data acquisition:

Address (Hex)	Register Name	Access	Description
0x00	DEVID_AD	Read	Analog Devices device ID — always reads 0xAD
0x01	DEVID_MST	Read	MEMS device ID — always reads 0x1D
0x02	PARTID	Read	Part ID for ADXL355 — reads 0xED
0x08	STATUS	Read	Data Ready, FIFO flags, NVM busy status
0x0E–0x10	XDATA3–1	Read	X-axis acceleration — 20-bit, MSB first
0x11–0x13	YDATA3–1	Read	Y-axis acceleration — 20-bit, MSB first

0x14–0x16	ZDATA3–1	Read	Z-axis acceleration — 20-bit, MSB first
0x28	FILTER	R/W	ODR and low-pass filter bandwidth selection
0x2C	RANGE	R/W	Measurement range: $\pm 2g$ / $\pm 4g$ / $\pm 8g$
0x2D	POWER_CTL	R/W	Standby/Measurement mode control
0x2F	RESET	Write	Write 0x52 to perform soft reset

8.1 SPI Transaction Protocol

Every SPI transaction with the ADXL355 begins with an address byte. The LSB of the address byte determines the operation:

- LSB = 0 → Write operation to the register
- LSB = 1 → Read operation from the register

Example: To read register 0x0E (XDATA3), send address byte 0x1D (= 0x0E << 1 | 0x01), then clock out the data bytes.

9. STM32 Firmware — Complete Implementation

The firmware is organized into modular functions covering SPI communication, sensor initialization, data reading, calibration, filtering, and tilt computation. All code runs on the STM32 HAL framework generated by STM32CubeMX.

9.1 Header File — adxl355.h

Define constants, register addresses, and function prototypes:

```
/* adxl355.h - ADXL355 Driver Header */
#ifndef ADXL355_H
#define ADXL355_H
#include "stm32f4xx_hal.h"
#include <math.h>

/* Register Addresses */
#define ADXL355_DEVID_AD 0x00
#define ADXL355_STATUS 0x04
#define ADXL355_XDATA3 0x08
#define ADXL355_YDATA3 0x0B
#define ADXL355_ZDATA3 0x0E
#define ADXL355_FILTER 0x28
```

```

#define ADXL355_RANGE 0x2C
#define ADXL355_POWER_CTL 0x2D
#define ADXL355_RESET 0x2F

/* Configuration Values */
#define ADXL355_RANGE_2G 0x01
#define ADXL355_ODR_4Hz 0x0A /* ODR=4Hz, LPF=1Hz */
#define ADXL355_MEAS_MODE 0x00 /* Measurement mode (bit0=0)
*/ #define ADXL355_SCALE 3.9e-6f /* g per LSB at +-2g range */

/* CS Pin Macros (configured as PA4 in CubeMX) */
#define ADXL355_CS_LOW() HAL_GPIO_WritePin(ADXL355_CS_GPIO_Port,
ADXL355_CS_Pin, GPIO_PIN_RESET)
#define ADXL355_CS_HIGH() HAL_GPIO_WritePin(ADXL355_CS_GPIO_Port,
ADXL355_CS_Pin, GPIO_PIN_SET)

void ADXL355_Init(SPI_HandleTypeDef *hspi);
void ADXL355_ReadXYZ(SPI_HandleTypeDef *hspi, float *ax, float *ay, float
*az);
void ADXL355_Calibrate(SPI_HandleTypeDef *hspi, float *ox, float *oy, float
*oz);
void ADXL355_CalcTilt(float ax, float ay, float az, float *tx, float *ty);
#endif /* ADXL355_H */

```

9.2 SPI Low-Level Read/Write Functions

```

/* Write a single byte to an ADXL355 register */
void ADXL355_WriteReg(SPI_HandleTypeDef *hspi, uint8_t reg, uint8_t val)
{
    uint8_t tx[2] = { (reg << 1) | 0x00, val }; /* LSB=0 for write */
    ADXL355_CS_LOW();
    HAL_SPI_Transmit(hspi, tx, 2, HAL_MAX_DELAY);
    ADXL355_CS_HIGH();
}

```

```

/* Read N bytes starting from register address */
void ADXL355_ReadRegs(SPI_HandleTypeDef *hspi, uint8_t reg,
uint8_t *buf, uint8_t len) {
    uint8_t addr = (reg << 1) | 0x01; /* LSB=1 for read */
    ADXL355_CS_LOW();
    HAL_SPI_Transmit(hspi, &addr, 1, HAL_MAX_DELAY);
    HAL_SPI_Receive(hspi, buf, len, HAL_MAX_DELAY);
    ADXL355_CS_HIGH();
}

```

9.3 Sensor Initialization

```

/* Initialize ADXL355: reset, set range, ODR, start measurement */
void ADXL355_Init(SPI_HandleTypeDef *hspi) {
    uint8_t id;

```

```

/* Step 1: Soft reset */
ADXL355_WriteReg(hspi, ADXL355_RESET, 0x52);
HAL_Delay(10); /* Wait for reset to complete */

/* Step 2: Verify device ID (should return 0xAD) */
ADXL355_ReadRegs(hspi, ADXL355_DEVID_AD, &id, 1);
if (id != 0xAD) { Error_Handler(); } /* Wiring error if fails */

/* Step 3: Configure range to +-2g (best noise for tilt) */
ADXL355_WriteReg(hspi, ADXL355_RANGE, ADXL355_RANGE_2G);

/* Step 4: Configure ODR to 4 Hz (low ODR = lower noise) */
ADXL355_WriteReg(hspi, ADXL355_FILTER, ADXL355_ODR_4Hz);

/* Step 5: Enter measurement mode (clear standby bit) */
ADXL355_WriteReg(hspi, ADXL355_POWER_CTL, ADXL355_MEAS_MODE);
HAL_Delay(100); /* Stabilization time */
}

```

9.4 Reading Raw Acceleration Data

```

/* Read X, Y, Z axes and convert to float in g units */
void ADXL355_ReadXYZ(SPI_HandleTypeDef *hspi,
                    float *ax, float *ay, float *az) {
    uint8_t raw[9];
    int32_t ix, iy, iz;

    /* Read 9 bytes: 3 bytes each for X, Y, Z (auto-increment) */
    ADXL355_ReadRegs(hspi, ADXL355_XDATA3, raw, 9);

    /* Reconstruct 20-bit signed integers (MSB first, left-aligned) */
    ix = ((int32_t)raw[0] << 12) | ((int32_t)raw[1] << 4) | (raw[2] >>
4); iy = ((int32_t)raw[3] << 12) | ((int32_t)raw[4] << 4) | (raw[5]
>> 4); iz = ((int32_t)raw[6] << 12) | ((int32_t)raw[7] << 4) |
(raw[8] >> 4);

    /* Sign-extend from 20-bit to 32-bit */
    if (ix & 0x80000) ix |= 0xFFFF00000;
    if (iy & 0x80000) iy |= 0xFFFF00000;
    if (iz & 0x80000) iz |= 0xFFFF00000;

    /* Convert to g using 3.9 ug/LSB scale factor */
    *ax = (float)ix * ADXL355_SCALE;
    *ay = (float)iy * ADXL355_SCALE;
    *az = (float)iz * ADXL355_SCALE;
}

```

9.5 Offset Calibration Function

```

/* Calibrate: average N samples at flat reference to find offsets */
void ADXL355_Calibrate(SPI_HandleTypeDef *hspi,
                      float *ox, float *oy, float *oz) {
    float sx = 0, sy = 0, sz = 0, ax, ay, az;
    const int N = 256;

    for (int i = 0; i < N; i++) {
        ADXL355_ReadXYZ(hspi, &ax, &ay,
                        &az); sx += ax; sy += ay; sz += az;
        HAL_Delay(5); /* ~4Hz ODR: wait for new sample */
    }
    *ox = sx / N; /* X and Y should be ~0 g when flat */
    *oy = sy / N;
    *oz = (sz / N) - 1.0f; /* Z should be ~1 g when flat */
}

```

9.6 Low-Pass Filter (Exponential Moving Average)

```

/* IIR low-pass filter applied after calibration */
static float lpf_x = 0, lpf_y = 0, lpf_z = 0;
#define ALPHA 0.05f /* Smoothing factor: lower = more filtering */

void ADXL355_LowPassFilter(float ax, float ay, float az,
                          float *fx, float *fy, float *fz) {
    lpf_x = ALPHA * ax + (1.0f - ALPHA) * lpf_x;
    lpf_y = ALPHA * ay + (1.0f - ALPHA) * lpf_y;
    lpf_z = ALPHA * az + (1.0f - ALPHA) * lpf_z;
    *fx = lpf_x; *fy = lpf_y; *fz = lpf_z;
}

```

9.7 Tilt Angle Calculation

```

/* Compute tilt angles in degrees from filtered acceleration */
void ADXL355_CalcTilt(float ax, float ay, float az,
                    float *tilt_x, float *tilt_y) {
    *tilt_x = atan2f(ax, sqrtf(ay*ay + az*az)) * (180.0f / M_PI);
    *tilt_y = atan2f(ay, sqrtf(ax*ax + az*az)) * (180.0f / M_PI);
}

```

9.8 Main Loop — main.c

The complete measurement loop in main.c integrates all functions:

```

/* In main.c, after MX_SPI1_Init() and MX_USART2_UART_Init() */
float offset_x, offset_y, offset_z;
float ax, ay, az, fx, fy, fz;
float tilt_x, tilt_y;

```

```

/* 1. Initialize sensor */
ADXL355_Init(&hspil);

/* 2. Calibrate offsets (sensor must be perfectly flat) */
printf("Calibrating... keep sensor flat!\r\n");
ADXL355_Calibrate(&hspil, &offset_x, &offset_y, &offset_z);
printf("Offsets: X=%.4f Y=%.4f Z=%.4f\r\n",
       offset_x, offset_y, offset_z);

/* 3. Main measurement loop */
while (1) {
    /* Read raw data */
    ADXL355_ReadXYZ(&hspil, &ax, &ay, &az);

    /* Apply calibration offsets */
    ax -= offset_x;
    ay -= offset_y;
    az -= offset_z;

    /* Apply low-pass filter */
    ADXL355_LowPassFilter(ax, ay, az, &fx, &fy, &fz);

    /* Calculate tilt angles */
    ADXL355_CalcTilt(fx, fy, fz, &tilt_x,
                    &tilt_y);

    /* Output via UART */
    printf("Tilt X: %.3f deg | Tilt Y: %.3f deg\r\n",
          tilt_x, tilt_y);

    HAL_Delay(250); /* 4 readings per second */
}

```

10. Working Principle — MEMS Sensing

The ADXL355 is a capacitive MEMS accelerometer. Inside the chip, a micro-scale proof mass is suspended by spring-like polysilicon flexures. When the sensor is tilted, the gravitational force causes the proof mass to deflect, changing the differential capacitance between the moving mass and fixed electrode plates. This capacitance change is amplified and digitized by the on-chip sigma-delta ADC into a 20-bit output per axis.

Because gravity is a constant 1 g vector always pointing downward, tilting the sensor simply redistributes this vector across the three axes in a mathematically defined way. This is the fundamental principle enabling tilt measurement from acceleration data.

11. Data Conversion and Tilt Mathematics

11.1 Raw Count to Acceleration

$$\text{Acceleration (g)} = \text{Raw Count} \times 3.9 \times 10^{-6} \text{ g/LSB}$$

At $\pm 2\text{g}$ range: 1 g corresponds to approximately 256,000 LSB ($= 1 \div 3.9 \times 10^{-6}$). The 20-bit signed range covers -52,0000 to +524,287 counts.

11.2 Gravity Vector Decomposition

$$|g| = \sqrt{X^2 + Y^2 + Z^2} \approx 1 \text{ g (sanity check)}$$

11.3 Tilt Angle Formulas

$$\begin{aligned}\theta_x \text{ (pitch)} &= \text{atan2}(X, \sqrt{Y^2 + Z^2}) \times (180 / \pi) \\ \theta_y \text{ (roll)} &= \text{atan2}(Y, \sqrt{X^2 + Z^2}) \times (180 / \pi)\end{aligned}$$

11.4 Example

$X = 0.1 \text{ g}, Y = 0 \text{ g}, Z = 0.99 \text{ g} \rightarrow \theta_x = \text{atan2}(0.1, 0.99) \approx 5.76^\circ$

12. Calibration Methodology

Calibration is the most impactful step in achieving sub-degree accuracy. Three stages are applied sequentially:

12.1 Offset Calibration (Zero-g Bias Removal)

$$\begin{aligned}X_{\text{offset}} &= (1/N) \times \sum X_i \\ X_{\text{corrected}} &= X_{\text{measured}} - X_{\text{offset}}\end{aligned}$$

$N = 256$ samples recommended. Sensor must be held flat and stationary during calibration. Both X and Y offsets should be near 0 g; Z offset is referenced to 1 g.

12.2 Scale Factor Calibration

Place sensor at exactly 0° and $+90^\circ$ using a precision level. Compare measured axis output to expected 0 g and 1 g. Derive a correction multiplier if deviation exceeds 1%.

12.3 Temperature Compensation

$$X_{\text{compensated}} = X_{\text{corrected}} - k \times (T - T_0)$$

k = temperature coefficient (characterize during testing), T = current temperature from ADXL355 TEMP register, T₀ = calibration reference temperature.

13. Noise Reduction Techniques

13.1 Sample Averaging

$$\text{Noise}_{\text{rms}} = \text{Noise}_{\text{single}} / \sqrt{N}$$

N = 256 samples reduces noise by 16×. At 4 Hz ODR this takes ~64 seconds per averaged output — appropriate for slow structural monitoring. For faster response, reduce N to 16 or 64.

13.2 Exponential Moving Average

$$Y[n] = \alpha \times X[n] + (1 - \alpha) \times Y[n-1]$$

α = 0.05 provides strong smoothing. α = 0.1–0.2 for slightly faster dynamic response. The IIR filter runs every sample with zero RAM overhead beyond three state variables.

13.3 Bandwidth Control via ODR

Setting ODR to 4 Hz in the FILTER register restricts the sensor’s measurement bandwidth, rejecting high-frequency vibration entirely at the hardware level before any digital processing.

13.4 Mechanical Isolation

The EVAL-ADXL355Z must be rigidly bolted or epoxied to the structure — loose mounting introduces spurious vibration. Use the four corner mounting holes with M2 screws and optional anti-vibration washers for outdoor deployments.

14. Precision Analysis — Achieving 0.01° Resolution

Parameter	Value
Gravity change at 0.01°	sin(0.01°) ≈ 174.5 μg
ADXL355 noise density	22.5 μg/√Hz

Effective noise @ 4 Hz ODR, N=256 avg	≈1.4 μg RMS
Signal-to-Noise Ratio	≈124× → 0.01° reliably detectable
Long-term drift	<5 μg over operating temperature range

CONCLUSION: The noise floor (~1.4 μg RMS after filtering) is more than 100× smaller than the 174.5 μg signal corresponding to 0.01° tilt. Sub-degree accuracy is physically achievable with this hardware.

15. Software and Development Tools

- STM32CubeMX — GUI-based peripheral configuration and HAL code generation
- STM32CubeIDE — Eclipse-based integrated development and debugging environment
- STM32 HAL Libraries — hardware abstraction layer providing portable SPI, GPIO, UART APIs
- ST-Link/V2 debugger — integrated on Nucleo board; used for flashing and live debugging
- Serial terminal (PuTTY / TeraTerm) — monitor UART output at 115200 bps

16. Hardware Implementation Summary

Component	Part	Purpose
MEMS Accelerometer	EVAL-ADXL355Z	Primary 3-axis sensing element
Microcontroller	STM32 Nucleo-F446RE	SPI master, processing, UART output
Power Supply	3.3 V from Nucleo CN6	Powers sensor VDDIO and VDD
SPI Bus	PA5/PA6/PA7 (SPI1)	SCLK, MISO, MOSI signals
Chip Select	PA4 (GPIO Output)	Software-controlled CS, active LOW
UART Debug	USART2 (PA2/PA3)	115200 bps terminal output
Mechanical Mount	Rigid bracket + M2 screws	Stable sensor attachment to structure

17. Hardware Architecture & Design Rational

The hardware architecture of the High-Precision Tilt Measurement System is designed to meet strict industrial reliability, sub-degree angular resolution ($\pm 0.01^\circ$), and long-term thermal stability standards. The system integrates an STM32L053R8 ultralow-power microcontroller with an Analog Devices ADXL355Z high-precision 3-axis MEMS accelerometer board via SPI1 protocol.

Component Specifications & Design Selection Rationale

- **STM32L053R8 Microcontroller:** Selected for its 32-bit ARM Cortex-M0+ architecture operating up to 32 MHz with 64 KB Flash and 8 KB SRAM. Provides hardware SPI1 for high-speed sensor interfacing, hardware USART2 for telemetry streaming, ultra-low power consumption, and robust interrupt architecture essential for real-time industrial monitoring.
- **ADXL355 Tri-Axis Digital Accelerometer:** Features 20-bit internal ADC resolution, ultra-low noise density ($\sim 22.5 \mu\text{g}/\sqrt{\text{Hz}}$), selectable $\pm 2\text{g}/\pm 4\text{g}/\pm 8\text{g}$ measurement ranges, integrated FIFO, and digital SPI output. Outperforms conventional 12-bit/16-bit analog sensors in resolution and EMI immunity.
- **Why SPI Communication Protocol:** SPI (Serial Peripheral Interface) Mode 0 (CPOL=0, CPHA=0) was selected over I2C due to its full-duplex synchronous operation, up to 10 MHz clock speeds, zero bus arbitration overhead, and deterministic timing. This guarantees loss-free 9-byte burst reads of 20-bit raw data.
- **Why UART Serial Telemetry:** USART2 operates at 115200 baud (8N1) via the ST-Link Virtual COM Port. UART provides a cross-platform asynchronous stream that connects seamlessly to desktop dashboards, microcontrollers, or wireless industrial gateways without proprietary drivers.
- **Why 3.3V System Voltage:** Both the STM32L053R8 MCU and ADXL355 sensor operate natively on 3.3 V DC (VDD and VDDIO 2.25 V – 3.6 V range). Direct 3.3 V rail powering eliminates active bidirectional logic level shifters, maximizing noise immunity and reliability.
- **Why Low-Noise Sensor Architecture:** A noise floor of $22.5 \mu\text{g}/\sqrt{\text{Hz}}$ corresponds to $\sim 1.4 \mu\text{g}$ RMS noise after 256-sample averaging and low-pass filtering. This permits reliable detection of micro-g accelerations and tilt angles down to $\pm 0.005^\circ$ to $\pm 0.01^\circ$, vital for Structural Health Monitoring (SHM).
- **Why MEMS Technology:** Surface-micromachined capacitive polysilicon proof-mass flexure technology provides solid-state durability, high shock rating (10,000g), compact silicon footprint, and exceptional thermal stability without mechanical wear.
- **Why Digital Sensor Output:** On-chip 20-bit sigma-delta ADCs digitize proof-mass deflection right at the sensor core, completely eliminating external analog noise pick-up, PCB trace crosstalk, and ADC quantization loss.
- **Why EVAL-ADXL355Z Evaluation Board:** The official EVAL-ADXL355Z board (UG-1030) features factory-installed $0.1 \mu\text{F}$ X7R decoupling capacitors, $1 \text{ k}\Omega$ isolation resistors, ground plane copper pours, and dual 6-pin headers (P1 & P2) designed specifically for low-noise sensor evaluation.

End-to-End System Signal Flow

Sensor (ADXL355 MEMS Capacitive Deflection) → SPI (Mode 0 20-bit Burst Read) → STM32 (HAL Data Acquisition) → Firmware (Sign Extension, g-Conversion, Filtering) → UART (USART2 115200 Baud Telemetry) → Python Dashboard (SerialReader Parser) → Degree Calculation (atan2 Trigonometry & Calibration) → Industrial Monitoring (Graphs, 3D Rotation, Gauges, FFT, Playback)

Pin Mapping & Hardware Wiring Reference Tables

Table 1: STM32 Pin Mapping

STM32	Function	ADXL355
PA4	Chip Select	P2 Pin 1
PA5	SPI Clock	P2 Pin 2
PA6	SPI MISO	P2 Pin 3
PA7	SPI MOSI	P2 Pin 4
3.3V	Power	P1 Pin 1
3.3V	Power	P1 Pin 3
GND	Ground	P1 Pin 5

Table 2: ADXL355 Header P1

Pin	Signal	Connection	Description
1	VDDIO	3.3V	Digital Supply
2	INT1	NC	Unused
3	VDD	3.3V	Main Supply
4	INT2	NC	Unused
5	GND	Ground	Common Ground
6	DRDY	NC	Unused

Table 3: ADXL355 Header P2

Pin	Signal	STM32	Description
1	CS	PA4	Chip Select
2	SCLK	PA5	SPI Clock
3	MISO	PA6	Master In Slave Out
4	MOSI	PA7	Master Out Slave In
5	NC	Unused	Reserved
6	NC	Unused	Reserved

Table 4: Complete Wiring Summary

Wire Colour	STM32 Pin	ADXL355 Pin	Signal	Direction	Purpose
Red	3.3V (CN6 Pin 4)	P1 Pin 1 (VDDIO)	VDDIO	Input	Digital Supply Power
Red	3.3V (CN6 Pin 4)	P1 Pin 3 (VDD)	VDD	Input	Main Analog/Sensor Power
Black	GND (CN6 Pin 6)	P1 Pin 5 (GND)	GND	Input	Common Ground Reference
Yellow	PA4 (CN7 Pin 17)	P2 Pin 1 (CS)	CS	Output (MCU->Sensor)	SPI Chip Select (Active LOW)
Orange	PA5 (CN10 Pin 11)	P2 Pin 2 (SCLK)	SCLK	Output (MCU->Sensor)	SPI Serial Clock Line
Green	PA6 (CN10 Pin 13)	P2 Pin 3 (MISO)	MISO	Input (Sensor->MCU)	Master In Slave Out Data Line
Blue	PA7 (CN10 Pin 15)	P2 Pin 4 (MOSI)	MOSI	Output (MCU->Sensor)	Master Out Slave In Data Line

18. STM32 Firmware Architecture & Code Implementation

The firmware is developed using the STM32 HAL (Hardware Abstraction Layer) framework. It implements hardware drivers for SPI1 master mode communication, GPIO CS pin control, 20-bit raw data burst reading, two's complement sign extension, raw-to-g acceleration scaling, and packetized USART2 telemetry streaming.

main.c — System Entry & Application Loop

Main application entry point. Initializes system clock via `SystemClock_Config()`, configures GPIO pins (PA4 CS), initializes SPI1 and USART2 peripherals, calls `ADXL355_Init()` to reset and configure the sensor to $\pm 2g$ range. In the main infinite loop, it verifies sensor identity by reading `DEVID_AD` (0xAD), invokes `ADXL355_ReadXYZ()` for 9-byte burst reads, formats the 20-bit raw values into a telemetry packet string 'ID=AD,X=...,Y=...,Z=...\r\n', and transmits it over USART2 at 115200 baud.

adxl355.c & adxl355.h — Sensor Driver Module

Implements low-level SPI transaction protocols and ADXL355 register control functions:

- `ADXL355_SignExtend20(uint32_t raw)`: Sign-extends 20-bit unsigned raw data into signed `int32_t`. Checks bit 19 (sign bit); if set, ORs with `0xFFFF0000`.
- `ADXL355_WriteRegister(reg, val)`: Asserts PA4 CS LOW, transmits address byte `((reg << 1) | 0x00)`, writes value byte, deasserts CS HIGH.
- `ADXL355_ReadRegister(reg)`: Asserts CS LOW, transmits address byte `((reg << 1) | 0x01)`, receives data byte, deasserts CS HIGH.
- `ADXL355_Reset()`: Writes `0x52` to RESET register (`0x2F`) and waits 100 ms.
- `ADXL355_Init()`: Deasserts CS, calls reset, writes RANGE register (`0x2C`) to $\pm 2g$ ($3.9 \mu g/LSB$), and sets `POWER_CTL` (`0x2D`) to `0x00` (measurement mode).
- `ADXL355_ReadXYZ(ADXL355_Data*)`: Performs a single-transaction 9-byte SPI burst read starting at

XDATA3 (0x08). Assembles 20-bit raw integers: $\text{raw} = (\text{DATA3} \ll 12) \mid (\text{DATA2} \ll 4) \mid (\text{DATA1} \gg 4)$, then applies `ADXL355_SignExtend20()`.

stm32l0xx_hal_msp.c — MCU Peripheral Support Package

Contains low-level hardware initialization hooks: `HAL_SPI_MspInit()` enables SPI1 clock (`__HAL_RCC_SPI1_CLK_ENABLE`) and configures PA5 (SCK), PA6 (MISO), and PA7 (MOSI) as Alternate Function 0 (AF0). `HAL_UART_MspInit()` enables USART2 clock and configures PA2 (TX) and PA3 (RX) as AF4.

stm32l0xx_it.c & system_stm32l0xx.c — Interrupts & System Clock

Provides system interrupt vector handlers (`SysTick_Handler` for HAL timebase tick) and system clock initialization (`SystemInit()`).

STM32CubeMX Peripheral Configuration Summary

- SPI1: Mode = Full-Duplex Master, Data Size = 8-bit, CPOL = Low (0), CPHA = 1Edge (0) [SPI Mode 0], NSS = Software, Prescaler = 32.
- USART2: Mode = Asynchronous, Baud Rate = 115200 bps, Word Length = 8 Bits, Parity = None, Stop Bits = 1.
- GPIO: PA4 configured as Output Push-Pull, High Speed, Output Level High (CS Idle).

19. Python Industrial Dashboard Architecture & Software Engineering

The desktop monitoring system is built using Python 3, PyQt5, pyqtgraph, PyOpenGL, NumPy, and ReportLab. It implements an industrial-grade dark/light themed architecture designed for real-time monitoring, FFT frequency analysis, session recording/playback, automated reporting, and interactive 3D visualization.

main.py — Application Window & Central Controller

Houses the main TiltMonitor QMainWindow. Features a 60px HeaderBar with live system clock, connection LED indicator, 6 live ParameterCards (Roll, Pitch, Accel X/Y/Z, Temp), collapsible Sensor Information panel, Peak Detection tracker, Session Info, Health Badge, and multi-tab central view container.

plotter.py — High-Performance pyqtgraph Real-Time Plotting

Implements LiveAxisPlot and AccelerationPlot classes utilizing circular numpy array buffers (0–1000 samples) with fixed coordinate axes. Features semi-transparent crosshair InfiniteLine tracking with live coordinate overlay, and a plot toolbar supporting PNG screenshot capture, zoom/pan mode toggle, view reset, and grid visibility toggle.

serial_reader.py — Thread-Safe Serial Port Interface

QThread-based SerialReader module. Uses precompiled regular expressions (`ID=([A-Za-z0-9]+)`, `X=(-?\d+)`, `Y=(-?\d+)`, `Z=(-?\d+)`) to continuously parse UART packets in a non-blocking background thread, emitting structured dictionaries over Qt signals.

theme.py — Dual-Theme Industrial Engine

Centralized theme engine defining dark industrial palette constants (BACKGROUND #1a1a2e, SURFACE #16213e, ACCENT #0f3460, HIGHLIGHT #e94560) and light industrial palette constants. Provides `get_dark_palette()`, `get_light_palette()`, `get_light_stylesheet()`, and `get_light_tab_stylesheet()` for instant switching.

tilt3d.py — PyOpenGL Real-Time 3D PCB Visualization

OpenGL QGLWidget canvas rendering a textured 3D circuit board model. Applies pitch and roll rotation matrix transformations in real-time to provide intuitive spatial visualization of structural orientation.

gaugeview.py — Custom QPainter Instrument Gauges

Custom QWidget instrument view rendering an artificial horizon tilt gauge, roll dial indicator, pitch ladder, and health status meter.

config_manager.py — Calibration & Settings Persistence

Manages JSON-based persistent storage for zero-point calibration offsets (calibration.json) and application settings (settings.json).

notifications.py — Animated Toast System

Provides animated bottom-right toast notifications (ToastNotification) that slide into view and fade out after 3 seconds for connection, calibration, and recording events.

fft_analysis.py — Fast Fourier Transform Frequency Analysis

Real-time spectral analysis module. Computes FFT magnitudes on 1024-sample buffers using `numpy.fft.rfft`, displaying dominant frequency (Hz), peak magnitude, frequency resolution, and magnitude spectrum plots.

playback.py — CSV Data Replay Engine

Full playback controller allowing recorded CSV sessions to be loaded and replayed through all graphs, 3D views, and gauges at 0.5×, 1×, 2×, or 5× speeds with timeline slider jump capability.

event_markers.py — Graph Event Annotation System

Automatically annotates graphs with vertical dashed `InfiniteLine` markers for key events: Sensor Connect/Disconnect, Calibration, Recording, Playback, and Alarms.

report_generator.py — Automated Industrial Report Generator

Generates professional multi-page PDF reports in the `Reports/` folder containing project metadata, device info, statistics tables, embedded graph screenshots, FFT spectra, and sign-off placeholders.

20. Applications

Application Domain	Description
Structural Health Monitoring	Continuous tilt tracking of bridges, dams, towers, buildings
Bridge Analysis	Long-term settlement and deflection monitoring under live load
Industrial Alignment	Precision leveling of machinery and heavy equipment platforms
Robotics & Automation	Attitude control for robotic arms and autonomous platforms
Seismic Sensing	Ground tilt changes before and after seismic events
Geotechnical Monitoring	Slope stability and landslide early-warning systems

21. Dashboard Visual Verification & Screenshots



Figure 2: Real-Time Industrial Dashboard Interface (Dark Theme Mode) displaying Live Parameters, Fixed-Scale Graphs, 3D PCB Rotation, and Statistics.

22. Conclusion

This report documents the complete design, wiring, firmware implementation, and validation of a high-precision tilt measurement system using the ADXL355 MEMS accelerometer on the EVAL-ADXL355Z evaluation board, interfaced with an STM32 microcontroller over SPI.

Key outcomes achieved:

- Correct physical connection of EVAL-ADXL355Z Headers P1 and P2 to the STM32 Nucleo board
- Complete STM32CubeMX project configuration for SPI1 (Mode 0), GPIO Chip Select, and USART2
- Full firmware driver covering initialization, SPI register read/write, data reconstruction, calibration, filtering, and tilt computation
- Verified tilt angle resolution of 0.01° through noise analysis ($\text{SNR} > 100\times$)
- Modular, reusable code architecture suitable for production deployment

This system is production-ready for integration into structural health monitoring, industrial alignment, and geotechnical sensing deployments as specified by TDCoB Pvt. Ltd.

End of Report — ADXL355 + STM32 Tilt System | TDCoB Pvt. Ltd. | Rev. 2